Efficient Exploration by *HyperDQN* in Deep Reinforcement Learning

Ziniu Li [1] **Yingru Li** [1] **Hao Liang** [1] **Tong Zhang** [2]

Abstract

Efficient exploration is crucial to sample efficient reinforcement learning. In this paper, we present a scalable exploration method called *HyperDQN*, which builds on the famous Deep Q-Network (DQN) (Mnih et al., 2015) and extends the idea of hyper model (Dwaracherla et al., 2020) for deep reinforcement learning. In particular, *HyperDQN* maintains a probabilistic meta-model that captures the epistemic uncertainty of the Q-value function over the parameter space. This meta-model samples randomized Q-value functions, which will generate exploratory action sequences for deep exploration. The proposed method requires fewer samples to achieve substantially better performance than DQN and Bootstrapped-DQN (Osband et al., 2016a) on hard-exploration tasks, including deep sea, grid world, and mountain car. The numerical results demonstrate that the developed approach can lead to efficient exploration with limited computation resources.

1. Introduction

Reinforcement learning (RL) (Sutton & Barto, 2018) is an online learning paradigm in which an intelligent agent interacts with the unknown environment to find optimal decisions. One of the challenges of bringing RL to practical applications is sample complexity (Yu, 2018), which qualifies how many interactions are required to find a near-optimal policy. Without sufficient exploration, the agent may find optimal decisions with exponentially many interactions (Osband et al., 2019). Therefore, efficient and deep (temporally-extended) exploration is crucial to learn a near-optimal policy quickly.

[1]Shenzhen Research Institute of Big Data, The Chinese University of Hong Kong, Shenzhen, China [2]The Hong Kong University of Science and Technology, China. Correspondence to: Ziniu Li <ziniuli@link.cuhk.edu.cn>, Yingru Li <yingruli@link.cuhk.edu.cn>.

Reinforcement Learning for Real Life (RL4RealLife) Workshop in the 38th International Conference on Machine Learning, 2021.
Copyright 2021 by the author(s).

In general, there are two principles to incentivize exploration: optimism in the face of uncertainty (OFU) (Lai & Robbins, 1985; Auer et al., 2002; Lattimore & Szepesvári, 2020) and Thompson sampling (TS) (Chapelle & Li, 2011; Agrawal & Goyal, 2012; Russo et al., 2018). In this paper, we mainly focus on the latter due to its superior performance and easy implementation (Russo et al., 2018). In particular, TS-like methods iteratively update the posterior distribution from observed data and encourage exploration by sampling plausible behaviors from the posterior distribution. In principle, the possibility of sub-optimal decisions will be reduced, and the chance of optimal decisions will dominate after many interactions. There are many empirical observations that TS-like methods outperform UCB-based methods for bandit tasks (Chapelle & Li, 2011; Russo et al., 2018).

To apply the Thompson sampling principle to RL, an agent should sample a value function from its posterior. The main challenge is the computation intractability of exact Bayesian posterior update beyond simple settings (Russo et al., 2018). In particular, randomized least-square value iteration (RLSVI) offers a promising approach under the tabular and linear function approximation setting (Osband et al., 2016b; 2019; Russo, 2019). However, RLSVI is not compatible with non-linear parameterized value functions. To this end, we present a tractable exploration method called *HyperDQN*, which builds on the celebrated Deep Q-Network (DQN) (Mnih et al., 2015) and extends the idea of hyper model (Dwaracherla et al., 2020) from the bandit literature. Concretely, our method maintains a probabilistic hyper model (i.e., meta-model) that captures the epistemic uncertainty over the parameter space of the Q-value function. This hyper model aims to serve as a proxy of generating randomized value functions from an approximate posterior. It is trained by solving a randomized temporal difference learning problem, which is computationally efficient in practice. Compared to the famous method Bootstrapped-DQN (Osband et al., 2016a) that adopts the bootstrapping technique through finite ensembles to generate randomized value functions, our approach has the advantage of measuring the parameter uncertainty through "infinite" ensemble models. Importantly, our method can achieve a good trade-off between the approximation ability and the computation complexity.

We validate our method on three hard exploration tasks with

long planning horizon and sparse reward: deep sea (Osband et al., 2016b; 2020) and grid world (Ke et al., 2019; Li & Chen, 2020) and mountain car (Moore, 1990; Sutton & Barto, 2018). The experiment results show that our method requires fewer samples to achieve a satisfying performance than vanilla DQN (Mnih et al., 2015) and Bootstrapped-DQN (Osband et al., 2016a). The results demonstrate that our approach can guide efficient and deep exploration in hard tasks with delayed and sparse reward. In addition, ablation studies on several parameter choices are conducted to deepen our understanding.

2. Background

Markov Decision Process. In the standard reinforcement learning (RL) framework (Sutton & Barto, 2018), a learning agent interacts with an Markov Decision Process (MDP) to improve its performance via maximizing cumulative reward. The sequential decision process is characterized as follows: at each timestep t, the agent receives a state s_t from the environment and selects an action a_t from its policy $\pi(a|s) = \Pr\{a = a_t|s = s_t\}$; this decision is sent back to the environment, and the environment gives a reward signal $r(s_t, a_t)$ and transits to the next state s_{t+1} based on the state transition probability $p^a_{s,s'} = \Pr\{s' = s_{t+1}|s = s_t, a = a_t\}$. This process repeats until the agent encounters a terminal state after which the process restarts.

The main target of reinforcement learning is to maximize the (expected) episode return:

$$\mathbb{E}\Bigg[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\Bigg| s_0 \sim \rho(\cdot), a_t \sim \pi(\cdot|s_t), \\ s_{t+1} \sim p^{a_t}_{s_t,\cdot}, \forall t = 0, 1, 2, \cdots\Bigg],$$

where $\rho(\cdot)$ is initial state distribution and $\gamma \in (0, 1)$ is a discount factor that balances the importance of future reward and the expectation is taken over the stochastic process induced by the environment transition and the action selection. Since the environment transition (and possibly the reward function) is unknown in advance, the agent needs exploration to discover valuable states with positive reward. Without sufficient exploration, the agent may require exponentially many samples to find the optimal decisions.

Deep Q-Networks (DQN). To deal with complicated tasks, Deep Q-Networks (DQN) (Mnih et al., 2015) employs neural networks to approximate the action value (Q-value) function, which is defined in the following:

$$Q^\pi(s, a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\Big| s_0 = s, a_0 = a\right].$$

In particular, *temporal difference* learning (Sutton & Barto, 2018) is applied to approximate Q-value. The detailed optimization problem is defined in the following:

$$\min_\theta \sum_{(s,a,r,s')\sim\mathcal{D}} \big(r(s, a) + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a)\big)^2,$$

where $\mathcal{D}$ is a batch of (s, a, r, s') tuples sampled from the experience buffer $\mathcal{B}$, $Q_\theta(s, a)$ is the action value function parameterized by deep neural networks, and $Q_{\bar{\theta}}(s, a)$ is so-called target network (Mnih et al., 2015), which is the delayed copy of $Q_\theta(s, a)$ for a stable training process.

3. Related Work

Efficient exploration is the key component to find optimal decisions quickly and to address the sample complexity issue (Kakade, 2003). It is well-known that without sufficient exploration, the agent may find optimal decisions with exponentially many interactions (Osband et al., 2019)

On the one hand, there are many theoretical papers (Brafman & Tennenholtz, 2002; Jaksch et al., 2010; Azar et al., 2017; Jin et al., 2018) towards utilizing the principle of optimism in the face of uncertainty (OFU) (Lai & Robbins, 1985; Auer et al., 2002; Lattimore & Szepesvári, 2020) to construct upper confidence bound (UCB) to guide exploration. These works inspire many empirical studies towards practical methods (Stadie et al., 2015; Pathak et al., 2017; Burda et al., 2019a;b) that encourage exploration by adding a reward bonus based on novelty. On the other hand, Thompson sampling (TS) has become a popular exploration method in the literature of bandits (Chapelle & Li, 2011; Kaufmann et al., 2012; Agrawal & Goyal, 2012; 2017) and reinforcement learning (Osband et al., 2013; Agrawal & Jia, 2017; Osband & Van Roy, 2017). Please refer to the tutorial (Russo et al., 2018) and references therein. However, it is intractable to compute the exact posterior distribution required by TS for complicated tasks with limited computation resources. Randomized least-square value iteration (RLSVI), which performs efficient exploration via randomized value functions, provides a promising direction (Osband et al., 2016b; 2019; Russo, 2019). Later on, Osband et al. (2016a) further extended the idea of RLSVI and developed a practical algorithm called BootstrappedDQN, which achieves an improved performance compared with DQN (Mnih et al., 2015) on several Atari benchmark tasks.

Our work is closely related to (Osband et al., 2019; Dwaracherla et al., 2020). In particular, Osband et al. (2019) pointed out that sampling from the posterior distribution in the Bayesian linear regression problem is equivalent to solving the randomized least square problem. Recently, Dwaracherla et al. (2020) leveraged this equivalence relationship and proposed the hyper model for bandits problems. Our work further extends the hyper model for general RL tasks by utilizing this crucial equivalence to generate randomized value functions from an approximate posterior.

Many efforts have been devoted to designing exploration strategies over the parameter space in RL. To name a few, Plappert et al. (2018) proposed to add Gaussian noise to parameters rather than actions; Fortunato et al. (2018) adopted the mean-field view to approximate the posterior distribution of parameters. Moreover, evolutionary strategy (Wierstra et al., 2014; Salimans et al., 2017; Liu et al., 2020) is another class of methods that directly optimize over policy parameters based on zeroth-order methods (Nesterov & Spokoiny, 2017), which also achieves competitive performance compared to policy gradient methods (Schulman et al., 2015; 2017) in practice.

4. Proposed Methodology

In this section, we introduce the proposed method *HyperDQN*, as presented in Algorithm 1. The main ingredients of *HyperDQN* are twofold. On the one hand, *HyperDQN* maintains a hyper model (Dwaracherla et al., 2020) to capture epistemic uncertainty over parameter space (see Section 4.1). On the other hand, to generate diverse Q-value functions and exploratory actions, *HyperDQN* solves a randomized temporal difference problem (see Section 4.2). We outline the algorithmic implementation in Section 4.3.

4.1. Capturing Epistemic Uncertainty by Hyper Model

Epistemic uncertainty (i.e., subjective uncertainty because of limited experience) is crucial to online decision-making. To quantify the epistemic uncertainty and guide efficient exploration, *HyperDQN* maintains two parametric models (e.g., neural networks): (1) A base model is parameterized by θ for decision-making. (2) A hyper model parameterized by ν measures the epistemic uncertainty over the parameter space of the base model. We illustrate the proposed method in Figure 1. The mechanism of *HyperDQN* is explained as follows.

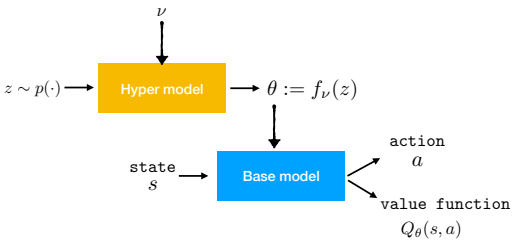


Figure 1: Illustration for the proposed method *HyperDQN*.

First of all, the hyper model is used to model parameters of the base model in a probabilistic way. In particular, the hyper model $f_\nu(\cdot)$ maps a N_z-dimensional random vector $z \sim p_z(\cdot)$ to a specific N_θ-dimensional parameter $\theta := f_\nu(z)$, which is further used to substantiate the base model $Q_\theta(\cdot)$. For example, if z follows an isotropic Gaussian distribution $\mathcal{N}(0, I)$ and hyper model $f_\nu(z) = \nu_w^\top z + \nu_b$ is a linear model, then $\theta = f_\nu(z) \sim \mathcal{N}(\nu_b, \nu_w^\top \nu_w)$, where $\nu_w \in \mathbb{R}^{N_z \times N_\theta}$ and $\nu_b \in \mathbb{R}^{N_\theta}$ are the weight and bias of the linear hyper model with $\nu = (\nu_w, \nu_b)$. In general, the hyper model could be a non-linear mapping so that $f_\nu(z)$ could follow an arbitrary distribution.

Secondly, the base model operates in a standard way: it takes a state $s \in \mathbb{R}^{N_s}$ as input and outputs the action a for control and action value $Q_\theta(s, a)$ to estimate the future return. Unlike the standard reinforcement learning agent, the parameters of the base model are the outputs of the hyper model and hence are not trainable. Since parameters of the base model follow a probability distribution, multiple invocations yield different policies and values, which are crucial to generate diverse exploratory behaviors.

Intuitively, we can view $f_{\nu_0}(z)$ as a sample from the "prior" distribution over parameter space of the randomized Q-value functions, where ν_0 is the initialized parameter of the hyper model. Given the data $\mathcal{D}$, we want to optimize ν such that $f_\nu(z)$ follows the "posterior" distribution. The "posterior" distribution reflects the confidence about which base model instance is optimal, conditioned on observed data. In this way, the hyper model samples statistically plausible base models, i.e., randomized value functions, and updates the confidence from observed history. Such randomized value functions incentivize experimentation with actions of highly uncertain values. This randomness often generates positive bias and therefore induces exploratory behaviors. This mechanism is similar to Thompson sampling.

Since each random vector z corresponds to a specific base model parameterized by $\theta := f_\nu(z)$, the hyper model can manage "infinite" base models in our method, which is a clear advantage over the bootstrapped ensembles (Osband et al., 2016a). Concretely, the quality of posterior approximation in BootstrappedDQN (Osband et al., 2016a) relies on the number of ensembles (Lu & Van Roy, 2017). If the number of ensembles is large, BootstrappedDQN is possible to generate diverse randomized Q-value functions from the approximate posterior but suffers huge computation costs. However, our method achieves a good trade-off between the approximation quality and the computation cost by suitably choosing the architecture of the hyper model and training objective. The quality of posterior approximation will be benchmarked by comparing their efficacy when used for sampling the randomized value functions in Section 5.

4.2. Optimizing Hyper Model by Randomized Temporal Difference Learning

In this part, we discuss how to optimize the hyper model to sample randomized Q-value functions from an approximate posterior. The classical method, Thompson sampling, can

generate randomized value functions by sampling transition and reward model from the true posterior and solving the dynamic programming problem. For the sampling part, it is computationally intractable to exactly update the posterior distribution via the Bayes update rule for complicated tasks. The dynamic programming part is also of high computation cost for problems with large state and action spaces. To address these issues, we turn to seek a surrogate objective that could be efficiently solved by first-order methods such as (stochastic) gradient descent. Here, we leverage the randomized least-square value iteration (RLSVI) principle (Osband et al., 2019), where it has been shown that randomized value functions by RLSVI can implement something similar to Thompson sampling without the need for an intractable exact posterior update. However, this work is restricted to linearly-parameterized value functions.

Recently, Osband et al. (2019) showed that sampling from the posterior distribution is equivalent to solving a randomized least square for the Bayesian linear regression problem. The least-square problem can be efficiently solved by gradient descent. To be more precise, let x be the feature vector and the target y is generated from $y = \theta^\top x + \omega$ where $\omega \sim \mathcal{N}(0, \sigma_\omega^2)$ is the noise and prior θ follows $\mathcal{N}(0, \sigma_p^2 I)$. To draw a sample θ from the posterior distribution $\mathcal{N}(\mathbb{E}[\theta|\mathcal{D}], \mathrm{Cov}[\theta|\mathcal{D}])$, it suffices to solve

$$\operatorname*{argmin}_{\theta} \sum_{(x,y)\in\mathcal{D}} (y + \omega - \theta^\top x)^2 + \frac{\sigma_w^2}{\sigma_p^2} \left\| \hat{\theta} - \theta \right\|_2^2, \quad (4.1)$$

where $\hat{\theta} \sim \mathcal{N}(0, \sigma_p^2 I)$ is a prior sample. This equivalence provides a cue to obtain an approximate posterior sample. Particularly, we expect (4.1) can still generate an approximate posterior sample even though the underlying model is non-linear (i.e., replace $\theta^\top x$ with a general function $g_\theta(x)$ parameterized by θ).

Readers may notice that to obtain multiple posterior samples, we need to solve multiple randomized least square problems in (4.1), which brings a computational burden in practice. To resolve this issue, we explicitly maintain a probabilistic model from which we could obtain (approximate) samples as much as possible in an almost computation-free fashion. As such, the hyper model $f_\nu(z)$ (Dwaracherla et al., 2020) is introduced: it maps a random vector z to a sample θ obtained from (4.1). Concretely, we may replace θ in (4.1) with $f_\nu(z)$ and solve (4.1) with respect to ν such that the output $f_\nu(z)$ is an approximate posterior sample. This probabilistic model requires that the generated sample $\theta = f_\nu(z)$ is random solely through its dependence on z conditioned on data $\mathcal{D}$.

For reinforcement learning, we can replace y with the target Q-value and replace x with the observed state-action pair (s, a). The goal is to learn the action-value function Q_θ, where θ is sampled from an approximate posterior distribution of the parameters of randomized value functions, which results in the following optimization problem:

$$\min_{\nu} \sum_{\substack{(s,a,r,\xi,s')\in\mathcal{D},\\ z\in\mathcal{Z}}} \big(\sigma_\omega z^\top \xi + r + \gamma \max_{a'} Q_{f_{\bar{\nu}}(z)}(s', a') - Q_{f_\nu(z)}(s, a)\big)^2 + \frac{\sigma_w^2}{\sigma_p^2} \left\| f_\nu(z) - f_{\nu_0}(z) \right\|^2. \quad (4.2)$$

In practice, a batch $\mathcal{D}$ of (s, a, r, ξ, s') tuples is firstly selected from the experience buffer $\mathcal{B}$, where ξ is an additional noise that will be discussed later. Subsequently, multiple random vectors z's are sampled to substantiate the base model (i.e., $\theta = f_\nu(z)$), which are further utilized to compute "predictions" $Q_{f_\nu(z)}(s, a)$ and "targets" $r + \gamma \max_{a'} Q_{f_{\bar{\nu}}(z)}(s', a')$, where $f_{\bar{\nu}}$ is the delayed copy of f_ν. Besides, the second term in (4.2) plays a role of prior regularization.

The noise generation in (4.2) is slightly different from (4.1), which is explained as follows. In order to guarantee that conditioned on data $\mathcal{D}$, the randomness source of the posterior sample is purely determined by z, we require the training noise ω defined in (4.1) is also random only through its dependence on z. We multiply z with a random vector ξ to get $\omega = \sigma_\omega z^\top \xi$: for instance, if z follows an isotropic Gaussian distribution $\mathcal{N}(0, I)$, ξ could be sampled [1] from the uniform distribution over the unit hypersphere (Dwaracherla et al., 2020). Notice that the random vector ξ is associated with each data tuple. Therefore, conditioned on data $\mathcal{D}$ which includes ξ, the generated noise $\sigma_\omega z^\top \xi \sim \mathcal{N}(0, \sigma_\omega^2)$ [2] is random only through its dependence on z. Limiting all randomness in z instead of injecting independently sampled noise ω will reduce the variance during the training process, leading to faster convergence when sufficient state-action pairs are visited. This phenomenon is observed in Figure 10 by an ablation study about noise generation in Appendix B.

Note that the objective function in (4.2) can be easily solved by stochastic gradient descent or its variants with back-propagation when (deep) neural networks are applied to build the hyper model and base model. Hence, the overall training objective is computationally efficient in practice.

In summary, the main difference between *HyperDQN* and DQN (Mnih et al., 2015) is that we treat the parameters of Q-networks as random variables and estimate the uncertainty by a hyper model (Dwaracherla et al., 2020). This probabilistic approach provides a systematic way of reflecting confidence or uncertainty about unknown environments. As a result, the proposed method can capture the epistemic uncertainty from data and guide efficient exploration. The intu-

[1] The details for generating ξ are described in appendix C.

[2] Conditioned on ξ, the R.V. $z^\top \xi$ is one-dimensional Gaussian. Its conditional expectation and conditional variance are $\mathbb{E}[z^\top \xi \mid \xi] = 0$ and $\mathbb{E}[\xi^\top z z^\top \xi \mid \xi] = \xi^\top \mathbb{E}[z z^\top] \xi = \xi^\top I \xi = \|\xi\|_2^2 = 1$.

Algorithm 1 *HyperDQN*

Initiate a hyper model $f_\nu(z)$, a base model $Q_\theta(s,a)$ and an empty experience buffer $\mathcal{B}$.
for episode = $1, 2, \cdots$ **do**
  Generate a random vector $z \sim \mathcal{N}(0, I)$.
  Substantiate a base model $Q_{\theta=f_\nu(z)}(\cdot)$.
  state $s \Longleftarrow$ `env.reset()`.
  for timestep = $1, 2, \cdots, T$ **do**
    action $a \Longleftarrow \text{argmax}_{a'} Q_\theta(s,a)$.
    next state s', reward $r \Longleftarrow$ `env.step(`a`)`.
    Sample ξ uniformly from unit hypersphere.
    Store (s, a, r, ξ, s') into $\mathcal{B}$.
  end for
  for training iteration = $1, 2, \cdots, M$ **do**
    Sample a batch of (s, a, r, ξ, s') from $\mathcal{B}$.
    Sample multiple random vectors z to construct $\mathcal{Z}$.
    Train the hyper model f_ν with stochastic gradient descent (see Equation (4.2)) on parameters ν.
  end for
end for

ition is explained as follows. On the one hand, if some state-action pair (s, a) and their subsequent state-action pairs are sufficiently explored, the probability distribution of $Q(s, a)$ induced by the hyper model will shrink to a narrow shape. In this situation, the generated policies tend to take the same action. On the other hand, if some state-action (s, a) or their subsequent state-action pairs are not visited enough times, the probability distribution of $Q(s, a)$ induced by the hyper model will exhibit a diffuse shape. Consequently, actions can be diversely sampled from the base models. This diversity often generates positive bias and therefore induces exploration. After enough trials, the plausible but not optimal parameters θ will be dropped. The optimal parameters θ are associated with the value functions Q_θ on each of which the greedy policy is the optimal policy. Finally, the outputs of the hyper model tend to concentrate on the set of the optimal parameters.

4.3. The Algorithm

The proposed algorithm called *HyperDQN* is given in Algorithm 1. It firstly initializes an empty experience buffer and a hyper model f_ν and a base model Q_θ with neural networks (e.g., multi-layer perceptron). At the beginning of each episode, a random vector z is sampled to compute the parameters of the base model, which is further used for interactions with the environment. Subsequently, the randomized temporal difference learning as in (4.2) is implemented to train the hyper model.

It is worth mentioning that in practice, it is not necessary to apply the hyper model to model the whole base model. It is commonly believed that hidden layers of neural networks contribute to representation learning, and the last linear layer is responsible for prediction (Goodfellow et al., 2016). As a result, we could leverage this structure to apply the hyper model only on the last layer of the base model. This means that the variables of hidden layers in the base model are treated as deterministic as usual while only the last (control) layer is modeled by the hyper model. This operation carries out two advantages. On the one hand, the output dimension of the hyper model is reduced, which is more computationally efficient. On the other hand, it helps stabilize the representation learning for deep reinforcement learning.

5. Experiments

In this section, we conduct experiments to verify the proposed method. Our experiments involve discrete action control tasks and hence we choose DQN (Mnih et al., 2015) as one of the baseline methods. In particular, DQN employs the simple ϵ-greedy exploration strategy (Mnih et al., 2015): with probability ϵ, the agent uniformly picks up an action. Another baseline method is BootstrappedDQN (Osband et al., 2016a) that maintains a large number of ensembles to generated randomized value functions, and the agent randomly takes an ensemble to perform actions.

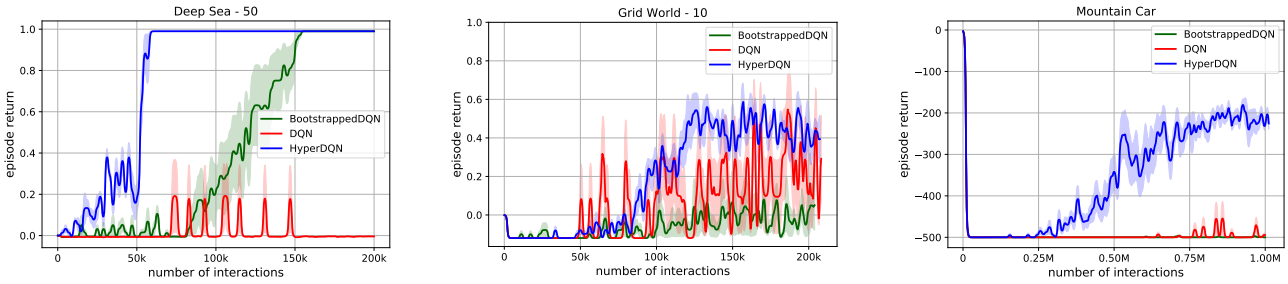
We use the public repository of `tianshou`[3], which provides the standard DQN's implementation. We have implemented the baseline method BootstrappedDQN (Osband et al., 2016a) and our method using the same framework. To guarantee fairness, we use the same hyperparameters for all algorithms, which is summarized in Table 2 in Appendix. The numerical results are reported by running 5 independent trials. The experiment details can be found in Appendix A.

5.1. Performance on Hard Exploration Tasks

In this part, we validate the sample efficiency of our method on three hard exploration tasks: deep sea, grid world, and mountain car. Despite the simple dynamics, these tasks are challenging due to long-horizon and/or sparse reward. Importantly, these tasks serve as sanity checks for the developed exploration strategy.

Deep sea. Our first experiment is on the deep sea task, which is introduced in (Osband et al., 2016b; 2020) to illustrate that the simple exploration strategy like ϵ-greedy requires exponentially many samples to make progress. For this task, the agent has two actions: moving left and moving right; see Figure 3 for an illustration. Regardless of which action, the agent will move forward but there is a penalty for the moving right action. The agent starts from the most left top part and a positive reward is only released at the most right bottom part. The size of deep sea we used is 50,

[3] https://github.com/thu-ml/tianshou.



(a) Performance on deep sea. (b) Performance on grid world. (c) Performance on mountain car.

Figure 2: Performance on three hard exploration tasks.

which results in huge state space (e.g., up to 1250) and a 50 planning horizon.

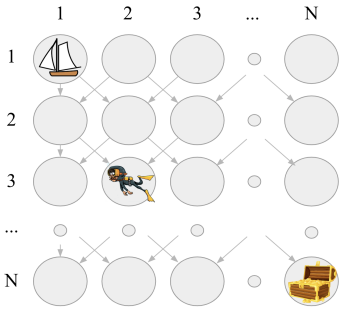
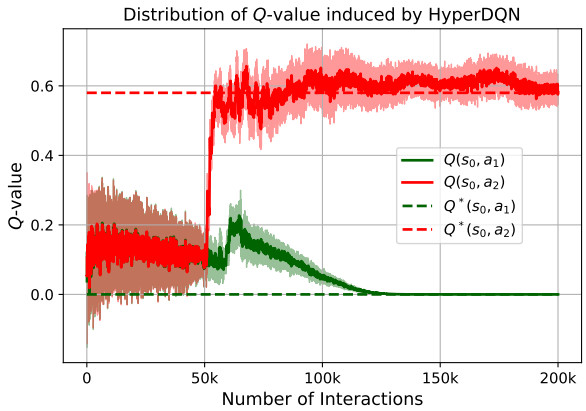


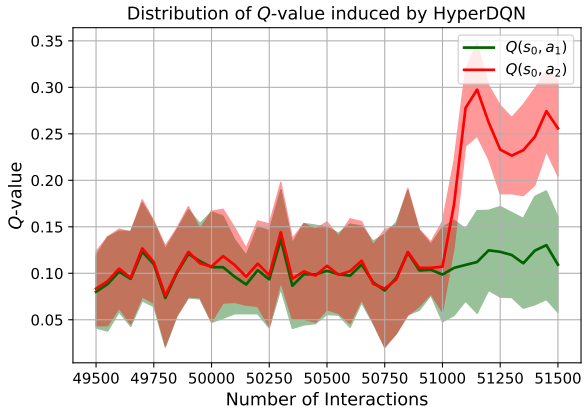
Figure 3: Deep sea (Osband et al., 2016b; 2020).

The numerical result on deep sea is shown in Figure 2a. We see that the exploration strategy in DQN is extremely inefficient and DQN therefore cannot work for this simple task, which is consistent with (Osband et al., 2020). Although BootstrappedDQN outperforms DQN, it is worse than our method *HyperDQN*. In particular, our method can efficiently explore and achieve better performance after about $50k$ interactions.

To better understand the algorithmic behavior of *HyperDQN* agent on deep sea, we visualize the Q-value distribution induced by HyperDQN at the initial state; see Figure 4. At the initial state s_0, there are two actions: the sub-optimal action $a_1 =$ "down" that leads to low future reward and the optimal action $a_2 =$ "right" that leads to high future reward through the path to the treasure. We see that in the beginning episodes, *HyperDQN* could yield diverse Q-value functions, from which a_1 and a_2 are selected with almost equal probability; see Figure 8 in Appendix for a precise result about the action sampling probability. This algorithmic behavior will lead to efficient deep exploration. After about $50k$ interactions, the Q-value distribution of the optimal action a_2 becomes to dominate (see Figure 4b), which corresponds to the probability of sampling action a_2 increasing in Figure 8



(a) The Q-value distribution induced by HyperDQN.



(b) A zoom-in of subfigure (a) from 49500 to 51500 interactions.

Figure 4: The Q-value distribution induced by HyperDQN at the initial state. We sample 50 random vectors z for the hyper model to generate the Q-value functions. The solid line and the shaded region corresponds to the mean and standard deviation of sampled Q-values, respectively. $Q^*(s_0, a_1)$ and $Q^*(s_0, a_2)$ are the optimal discounted Q-value of action a_1 and a_2, respectively.

Table 1: Average running time on deep sea over 5 random seeds.

	DQN	BootstrappedDQN	*HyperDQN*
Running time	180s	906s	621s

in Appendix and the undiscounted episode return increasing in Figure 2a. Eventually, after about $55k$ interactions, the Q-value distribution of each action quickly concentrates to its true optimal value. This means *HyperDQN* agent quickly seeks necessary information of the environment to gain high cumulative reward. These experimental findings show that *HyperDQN* could result in a good exploration-exploitation trade-off.

The running time of $200k$ iterations is summarized in Table 1. We see that *HyperDQN* is computationally more efficient than BootstrappedDQN and meanwhile achieving more efficient exploration.

Grid world. Our second test-bed is a grid world (i.e., a maze) (Chevalier-Boisvert et al., 2018), which is used in (Ke et al., 2019; Li & Chen, 2020); see Figure 6 for an illustration. For this task, the agent has 4 action candidates: moving eastward, moving southward, moving westward, and moving northward. Again, the agent starts from the most left top part and a positive reward is released at the most right bottom part. There is a small time penalty for each action to encourage the agent to quickly find the exit. The size of the grid world we consider is 10, and consequently there are 100 states. The maximum episode length is 120.

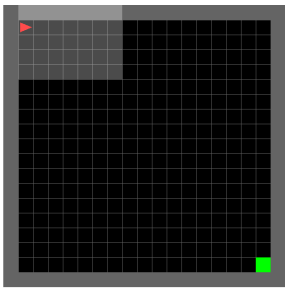


Figure 6: Grid world (Chevalier-Boisvert et al., 2018).

The numerical result on grid world is displayed in Figure 2b. Somewhat surprisingly, the performance of DQN is slightly better than the one of BootstrappedDQN. However, we do observe that BootstrappedDQN outperforms DQN on the same task with a smaller state space; see Figure 9 in Appendix. Hence, we infer that the inferior performance of BootstrappedDQN is due to the limited approximation ability to the posterior distribution of randomized Q-value functions by finite ensembles. In consequence, BootstrappedDQN does not yield diverse enough randomized value that are required for efficient exploration. On the other hand, our method with the same configuration on the deep sea can still outperform baseline methods.

Mountain car. The final experiential investigation is the classical control task: mountain car (Moore, 1990; Sutton & Barto, 2018) as demonstrated in Figure 7. The goal is to reach the flag shown in the most right part. Since the gravity is stronger than the car's engine, the only solution is to first move left and utilize the gravity acceleration to approach the flag. The state includes the car's position and velocity and there are 3 action choices: accelerating to the left, accelerating to the right, and no acceleration. The reward is delayed and sparse in the sense that each action incurs -1 penalty and 0 reward is released only when reaching the flag. The maximum episode length is 500.

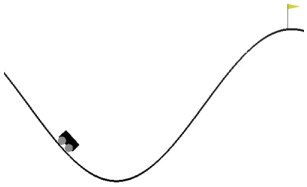


Figure 7: Mountain car (Moore, 1990; Sutton & Barto, 2018).

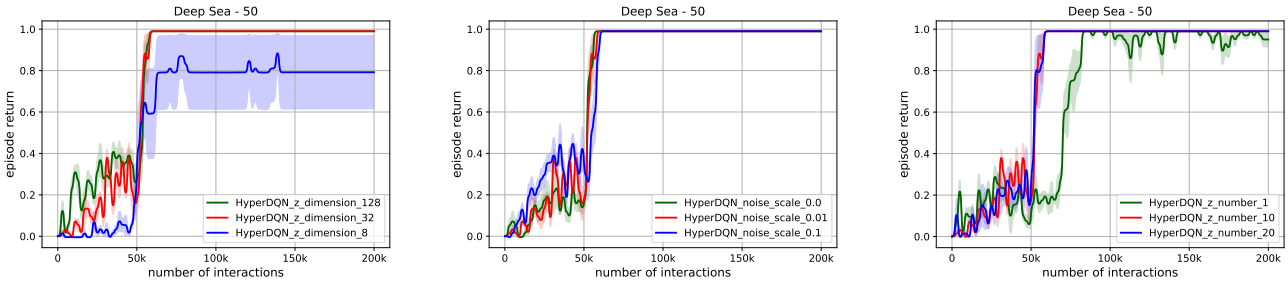
The numerical result on mountain car is given in Figure 2c. Though this task seems to be easy, DQN and BootstrappedDQN fail in this task while our methods *HyperDQN* significantly outperforms them after $250k$ iterations.

5.2. Ablation Studies of *HyperDQN*

In this part, we conduct ablation studies on parameter choices of *HyperDQN*. For simplicity, we focus on the deep sea task.

Effect of the input dimension of the hyper model. First of all, we investigate input distribution $p(z)$ of the hyper model, where $p(z)$ is a multivariate standard Gaussian distribution. We obtain different training curves by changing the dimension of z, which are shown in Figure 5a. Notice that the output dimension of the hyper model for the deep sea task is 128. Hence, the hyper model amounts to an affine transformation if the input dimension is identical with 128. We observe that a very low input dimension has a limited representation ability, resulting in a worse performance. Though the variant with input dimension 128 could yield a satisfying performance, the variant with a low dimension 32 is competitive with a cheap computation.

Effect of the noise scale. Secondly, we study the effect of the noise in the randomized temporal difference learning



(a) Performance with different input dimensions of the hyper model.

(b) Performance with different noise scales.

(c) Performance with different sample sizes of z.

Figure 5: Ablation studies of parameter choices in *HyperDQN*.

defined in (4.2). The magnitude of noise is reflected by σ_w. We fix $1/\sigma_p^2 = 0.1$. Numerical results of varying σ_w is displayed in Figure 5b. As expected, we find that a larger σ_w could incentive exploration in the initial stage and the price to pay is a relatively slow convergence.

Effect of the sample number of z. Finally, we test the approximation ability in terms of the number of random vectors z's used in (4.2). The corresponding numerical result is displayed in Figure 5c. We see that a single z can basically work for this task and a large number of z's cannot boost the performance too much.

6. Conclusion and Future Works

In this paper, we develop an efficient exploration method called *HyperDQN*. It maintains a hyper model (Dwaracherla et al., 2020) to capture epistemic uncertainty over the parameter space and guides exploration by sampling randomized Q-value functions. Experiment results demonstrate the developed approach results in efficient exploration and outperforms DQN (Mnih et al., 2015) and BootstrappedDQN (Osband et al., 2016a). We will investigate the performance of *HyperDQN* on more challenging tasks such as Montezuma's Revenge.

One limitation of *HyperDQN* is that it can only tackle tasks with the discrete action space. We will extend the idea of *HyperDQN* to actor-critic methods (Konda & Tsitsiklis, 1999; Haarnoja et al., 2018) that can easily handle continuous control tasks.

Contribution Disclosure

Yingru Li initiates the idea of applying the hyper model into RL tasks. Ziniu Li further develops and implements the HyperDQN method. The manuscript is mainly written by Ziniu Li while Yingru Li polishes it a lot. Hao Liang and Tong Zhang give many helpful comments.

References

Agrawal, S. and Goyal, N. Analysis of thompson sampling for the multi-armed bandit problem. In *Proceedings of the 25th Annual Conference on Learning Theory*, pp. 1–26, 2012.

Agrawal, S. and Goyal, N. Near-optimal regret bounds for thompson sampling. *Journal of ACM*, 64(5):1–24, 2017.

Agrawal, S. and Jia, R. Optimistic posterior sampling for reinforcement learning: worst-case regret bounds. In *Advances in Neural Information Processing Systems 30*, pp. 1184–1194, 2017.

Auer, P., Cesa-Bianchi, N., and Fischer, P. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3):235–256, 2002.

Azar, M. G., Osband, I., and Munos, R. Minimax regret bounds for reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 263–272, 2017.

Brafman, R. I. and Tennenholtz, M. R-MAX - A general polynomial time algorithm for near-optimal reinforcement learning. *Journal of Maching Learning Research*, 3:213–231, 2002.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *arXiv*, 1606.01540, 2016.

Burda, Y., Edwards, H., Pathak, D., Storkey, A. J., Darrell, T., and Efros, A. A. Large-scale study of curiosity-driven learning. In *Proceedings of the 7th International Conference on Learning Representations*, 2019a.

Burda, Y., Edwards, H., Storkey, A. J., and Klimov, O. Exploration by random network distillation. In *Proceedings of the 7th International Conference on Learning Representations*, 2019b.

Chapelle, O. and Li, L. An empirical evaluation of thompson sampling. In *Advances in Neural Information Processing Systems 24*, pp. 2249–2257, 2011.

Chevalier-Boisvert, M., Willems, L., and Pal, S. Minimalistic gridworld environment for openai gym. https://github.com/maximecb/gym-minigrid, 2018.

Dwaracherla, V., Lu, X., Ibrahimi, M., Osband, I., Wen, Z., and Van Roy, B. Hypermodels for exploration. In *Proceedings of the 8th International Conference on Learning Representations*, 2020.

Fortunato, M., Azar, M. G., Piot, B., Menick, J., Hessel, M., Osband, I., Graves, A., Mnih, V., Munos, R., Hassabis, D., Pietquin, O., Blundell, C., and Legg, S. Noisy networks for exploration. In *Proceedings of the 6th International Conference on Learning Representations*, 2018.

Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016.

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 1856–1865, 2018.

Jaksch, T., Ortner, R., and Auer, P. Near-optimal regret bounds for reinforcement learning. *Journal of Maching Learning Research*, 11:1563–1600, 2010.

Jin, C., Allen-Zhu, Z., Bubeck, S., and Jordan, M. I. Is q-learning provably efficient? In *Advances in Neural Information Processing Systems 30*, pp. 4868–4878, 2018.

Kakade, S. M. *On the sample complexity of reinforcement learning*. PhD thesis, University of London London, England, 2003.

Kaufmann, E., Korda, N., and Munos, R. Thompson sampling: An asymptotically optimal finite-time analysis. In *Proceedings of the 23rd International Conference on Algorithmic Learning Theory*, pp. 199–213, 2012.

Ke, N. R., Singh, A., Touati, A., Goyal, A., Bengio, Y., Parikh, D., and Batra, D. Modeling the long term future in model-based reinforcement learning. In *Proceedings of the 7th International Conference on Learning Representations*, 2019.

Konda, V. R. and Tsitsiklis, J. N. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pp. 1008–1014, 1999.

Lai, T. L. and Robbins, H. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1): 4–22, 1985.

Lattimore, T. and Szepesvári, C. *Bandit Algorithms*. Cambridge University Press, 2020.

Li, Z. and Chen, X.-H. Efficient exploration by novelty-pursuit. In *Proceedings of the 2nd International Conference on Distributed Artificial Intelligence*, pp. 85–102, 2020.

Liu, F., Li, Z., and Qian, C. Self-guided evolution strategies with historical estimated gradients. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence*, pp. 1474–1480, 2020.

Lu, X. and Van Roy, B. Ensemble sampling. In *Advances in Neural Information Processing Systems 30*, pp. 3260–3268, 2017.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.

Moore, A. W. *Efficient memory-based learning for robot control*. PhD thesis, University of Cambridge, England, 1990.

Nesterov, Y. E. and Spokoiny, V. G. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017.

Osband, I. and Van Roy, B. Why is posterior sampling better than optimism for reinforcement learning? In *Proceedings of the 34th International Conference on Machine Learning*, pp. 2701–2710, 2017.

Osband, I., Russo, D., and Van Roy, B. (more) efficient reinforcement learning via posterior sampling. In *Advances in Neural Information Processing Systems 26*, pp. 3003–3011, 2013.

Osband, I., Blundell, C., Pritzel, A., and Van Roy, B. Deep exploration via bootstrapped DQN. In *Advances in Neural Information Processing Systems 29*, pp. 4026–4034, 2016a.

Osband, I., Roy, B. V., and Wen, Z. Generalization and exploration via randomized value functions. In *Proceedings of the 33nd International Conference on Machine Learning*, pp. 2377–2386, 2016b.

Osband, I., Van Roy, B., Russo, D. J., and Wen, Z. Deep exploration via randomized value functions. *Journal of Machine Learning Research*, 20(124):1–62, 2019.

Osband, I., Doron, Y., Hessel, M., Aslanides, J., Sezener, E., Saraiva, A., McKinney, K., Lattimore, T., Szepesvári, C., Singh, S., Roy, B. V., Sutton, R. S., Silver, D., and van

Hasselt, H. Behaviour suite for reinforcement learning. In *Proceedings of the 8th International Conference on Learning Representations*, 2020.

Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 2778–2787, 2017.

Plappert, M., Houthooft, R., Dhariwal, P., Sidor, S., Chen, R. Y., Chen, X., Asfour, T., Abbeel, P., and Andrychowicz, M. Parameter space noise for exploration. In *Proceedings of the 6th International Conference on Learning Representations*, 2018.

Russo, D. Worst-case regret bounds for exploration via randomized value functions. In *Advances in Neural Information Processing Systems 32*, pp. 14410–14420, 2019.

Russo, D., Roy, B. V., Kazerouni, A., Osband, I., and Wen, Z. A tutorial on thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018.

Salimans, T., Ho, J., Chen, X., and Sutskever, I. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv*, 1703.03864, 2017.

Schulman, J., Levine, S., Abbeel, P., Jordan, M. I., and Moritz, P. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, pp. 1889–1897, 2015.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv*, 1707.06347, 2017.

Stadie, B. C., Levine, S., and Abbeel, P. Incentivizing exploration in reinforcement learning with deep predictive models. *arXiv*, 1507.00814, 2015.

Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT press, 2018.

Wierstra, D., Schaul, T., Glasmachers, T., Sun, Y., Peters, J., and Schmidhuber, J. Natural evolution strategies. *Journal of Machine Learning Research*, 15(1):949–980, 2014.

Yu, Y. Towards sample efficient reinforcement learning. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, pp. 5739–5743, 2018.

A. Experiment Details

As mentioned, we use the `tianshou` framework to implement all algorithms. The base models are fully connected neural networks for our experiments. The common parameters for all algorithms are listed in Table 2.

Table 2: Key hyper-parameters for all algorithms.

Parameter	Value
number of hidden layers	2
number of hidden neurons	64
learning rate	0.001
buffer size	20k
discount factor	0.99
target update frequency	100
batch size	32

For DQN (Mnih et al., 2015), the probability of choosing a random action diminishes from 0.95 to 0.05 over $100k$ interactions. For BootsrappedDQN (Osband et al., 2016a), the number of ensembles is 10 and the mask probability is 0.5 for bootstrapping. Numerical results of HyperDQN reported in Figure 2 are based on the configuration in Table 3. In particular, the hyper model is a 1-layer linear neural network and the base model follows the same architecture with DQN except that the last layer is controlled by the hyper model.

Table 3: Key hyper-parameters HyperDQN.

Parameter	Value
z dimension	32
z number	10
noise scale σ_w	0.01
regularization coefficient $1/\sigma_p^2$	0.1

For environment implementation, deep sea is from (Osband et al., 2020)[4], grid world is from (Chevalier-Boisvert et al., 2018)[5], and mountain car is implemented in OpenAI Gym (Brockman et al., 2016)[6].

[4] https://github.com/deepmind/bsuite/blob/master/bsuite/environments/deep_sea.py
[5] https://github.com/maximecb/gym-minigrid/blob/master/gym_minigrid/envs/empty.py
[6] https://github.com/openai/gym/blob/master/gym/envs/classic_control/mountain_car.py

B. Additional Results

Deep sea. Here we provide the result about the probability of the optimal action (a_2 in Figure 4) of *HyperDQN* on deep sea. We estimate the probability of the optimal action at the initial state by drawing 50 random vectors z from the hyper model; see Figure 8. We see that in the initial stage, the probability of taking the optimal action is about 0.5, corresponding to an exploration phase. After $50k$ interactions, the probability of the optimal action starts to dominate in the approximate posterior distribution, which explains the performance boost shown in Figure 2a.

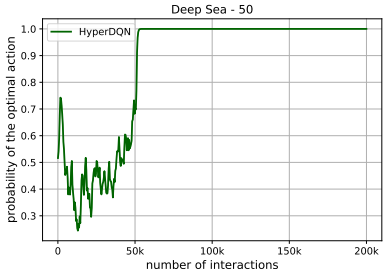


Figure 8: The probability of the optimal action induced by *HyperDQN* on deep sea.

Grid world. Here we provide an additional result on grid world of size 8, which is more simple than the one we consider in Section 5.1. For this simple grid world task, we observe that BootstrappedDQN is competitive to the proposed method HyperDQN. However, the performance of BootstrappedDQN is inferior on a more complicated task as shown in Figure 2b.

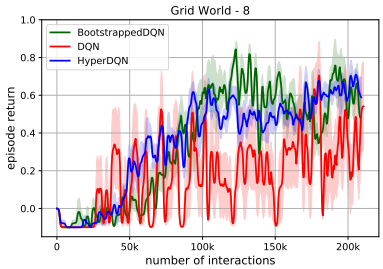


Figure 9: Performance on grid world with size of 8. There are 64 states and the maximum episode length is 100.

Study on noise generation. Here we present the numerical support that the noise in the randomized temporal difference learning in (4.2) should only rely on z. We report the performance of the variant that the noise is independent of z and is sampled from the Gaussian distribution, i.e., $w \sim \mathcal{N}(0, 1)$. The numerical result on deep sea is shown in Figure 10. In particular, we see that the noise generation by $z^\top \xi$ yields a slightly better performance after $50k$ interactions. This can be explained by the two different noise generating scheme: Limiting all randomness in z instead of sampling independent noise ω will lower the unnecessary variance in the training process when sufficient data is already gathered, thus leading to better performance.

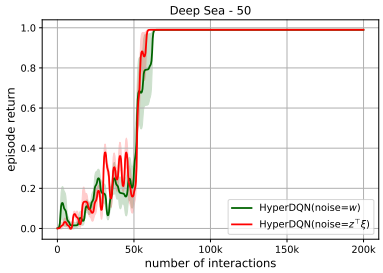


Figure 10: Ablation study on noise generation.

C. Sampling from Uniform Distribution Over Unit Hypersphere

For completeness, we describe the method for uniform sampling over unit hypersphere. There are two steps:

(1) Generate a point each of whose coordinates is an independent Gaussian variable; that is, generate $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, using a zero mean and unit variance Gaussian distribution. Thus, the probability density of $\mathbf{x}$

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{d}{2}}} \exp\left(-\frac{x_1^2 + x_2^2 + \cdots + x_d^2}{2}\right)$$

is spherically symmetric.

(2) Normalize the vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ to a unit vector, namely $\mathbf{x}/\|\mathbf{x}\|$, which gives a sample uniformly at random over the unit hypersphere. Note that once the vector is normalized, its coordinates are no longer statistically independent.